

Summary

`pg-ext-manager.sh` - a comprehensive tool for managing PostgreSQL extensions across all GitLab databases. This script provides visibility into extension versions and enables safe, batch updates with proper confirmation workflows.

Description

This tool addresses the need for systematic PostgreSQL extension management in our GitLab infrastructure. It provides a unified interface to monitor extension status across multiple databases and perform updates when needed.

Business Context and Operational Requirement

PostgreSQL extensions are critical components that extend the core functionality of the database system. When performing major or minor version upgrades of PostgreSQL within our Patroni clusters, the underlying extension packages are updated to versions compatible with the new PostgreSQL release. However, the extension objects within individual databases are **not automatically upgraded** during the PostgreSQL upgrade process.

This creates a critical operational gap: while the new extension binaries are available on the filesystem, the database catalogs continue referencing older extension versions until explicitly updated via `ALTER EXTENSION ... UPDATE` commands. This version mismatch can lead to:

- **Functional discrepancies** between expected and actual extension behavior
- **Security vulnerabilities** if critical patches in newer extension versions remain unapplied
- **Performance degradation** due to missing optimizations in updated versions
- **Operational risk** from running outdated extension code against newer PostgreSQL binaries
- **Compliance issues** during security audits when extension versions lag behind available updates

Integration with Database Upgrade Procedures

This script has been developed as a **mandatory post-upgrade verification and remediation tool** within our standardized database upgrade workflow. Following the completion of a Patroni cluster upgrade (either in-place or via switchover), this tool must be executed to:

1. **Audit Extension Status:** Immediately identify all extensions requiring updates across all databases in the newly upgraded cluster
2. **Systematic Update Execution:** Apply extension updates in a controlled, documented manner with built-in confirmation workflows

3. **Post-Update Verification:** Validate that all extensions are current and properly aligned with the PostgreSQL version
4. **Compliance Documentation:** Generate audit trails showing extension version status before and after upgrade operations

By incorporating this tool into our upgrade procedures, we ensure **version consistency**, **reduce technical debt**, and maintain **operational excellence** across our database infrastructure. This systematic approach eliminates the manual overhead of checking each database individually and reduces the likelihood of overlooking critical extension updates during the upgrade process.

Key Features

Multi-database Extension Reporting

- Scans all PostgreSQL databases in the GitLab instance
- Color-coded status indicators (`Current`, `Outdated`, `↑ Newer`) **Newer** in this context refers to if a `pg_` extension is higher version than that comes default with the package. This is possible when we download the extension directly from source and install them instead of relying on the postgresql debain packages.
- Consolidated summary statistics across all databases

Flexible Filtering Options

- View all extensions, only outdated, or only current extensions
- Database-by-database breakdown with version comparisons
- Quick identification of extensions requiring updates

Safe Update Mechanism

- Interactive confirmation before applying any changes
- Individual update tracking with success/failure reporting
- Displays error messages for troubleshooting failed updates
- Post-update verification with automatic status check

Developer-Friendly Output

- Generates copy-paste ready SQL commands for manual updates
- Clear, tabular format for easy reading
- Detailed version information (current vs. available)

Usage Examples

```
./pg-ext-manager.sh           # Show all extensions
./pg-ext-manager.sh outdated  # Show only outdated extensions
./pg-ext-manager.sh update    # Update all outdated extensions
```

Recommended Workflow for Database Upgrades

Following a PostgreSQL version upgrade on any Patroni cluster, execute the following sequence:

```
# 1. Audit current extension status across all databases
./pg-ext-manager.sh all
```

```
# 2. Identify extensions requiring updates
./pg-ext-manager.sh outdated
```

```
# 3. Review the proposed updates and apply them with confirmation
./pg-ext-manager.sh update
```

```
# 4. Verify all extensions are current post-update
./pg-ext-manager.sh all
```

This workflow ensures complete extension version alignment with the upgraded PostgreSQL release.

Benefits

- Reduces manual effort - No need to check each database individually
- Improves security - Ensures extensions are up-to-date with latest patches
- Prevents issues - Identifies version mismatches before they cause problems
- Audit trail - Clear reporting of what was updated and when
- Standardizes upgrade procedures - Provides consistent methodology across all database upgrades
- Mitigates operational risk - Ensures extension-to-PostgreSQL version compatibility
- Accelerates upgrade cycles - Reduces time spent on post-upgrade validation tasks

Testing

Tested on `patroni-pgtest-v18-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal`

```
vporalla@patroni-pgtest-v18-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:~$ /home/vporalla/pg_exten
```

```
Usage: /home/vporalla/pg_extension_script_colors.sh [option]
```

Options:

<code>all</code>	Show all extensions (default)
<code>outdated</code>	Show extensions that need updating
<code>current</code>	Show extensions that are up to date
<code>update</code>	Update all outdated extensions (requires confirmation)
<code>help</code>	Display this help message

Examples:

```
/home/vporalla/pg_extension_script_colors.sh # Show all extensions
```

```

/home/vporalla/pg_extension_script_colors.sh outdated # Show only outdated extensions
/home/vporalla/pg_extension_script_colors.sh current  # Show only current extensions
/home/vporalla/pg_extension_script_colors.sh update   # Update all outdated extensions

```

#Show All Extensions

vporalla@patroni-pgtest-v18-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:~\$ /home

PostgreSQL Extensions Report

Host: patroni-pgtest-v18-01-db-db-benchmarking

Filter: All extensions

Date: 2025-10-03 09:18

Database: postgres

Extension	Current	Available	Status
pg_stat_kcache	2.3.1	2.3.1	OK
pg_stat_statements	1.12	1.12	OK
pg_wait_sampling	1.1	1.1	OK
plpgsql	1.0	1.0	OK

Database: gitlabhq_production

Extension	Current	Available	Status
pg_repack	1.5.2	1.5.0	↑ NEWER
btree_gin	1.3	1.3	OK
dblink	1.2	1.2	OK
pg_trgm	1.6	1.6	OK
pg_wait_sampling	1.1	1.1	OK
pgstattuple	1.5	1.5	OK
plpgsql	1.0	1.0	OK
amcheck	1.4	1.5	OUTDATED
btree_gist	1.7	1.8	OUTDATED
pageinspect	1.12	1.13	OUTDATED
pg_buffercache	1.5	1.6	OUTDATED
pg_stat_statements	1.11	1.12	OUTDATED

Quick Summary

```

postgres:                Total:  4 | Current:  4 | Outdated:  0
gitlabhq_production:      Total: 12 | Current:  6 | Outdated:  5
OVERALL:                  Total: 16 | Current: 10 | Outdated:  5

```

```

#Show Outdated Extensions
vporalla@patroni-pgtest-v18-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:~$ /home
=====

PostgreSQL Extensions Report
Host: patroni-pgtest-v18-01-db-db-benchmarking
Filter: Outdated extensions
Date: 2025-10-03 09:18
=====

Database: gitlabhq_production
-----

```

Extension	Current	Available	Status
amcheck	1.4	1.5	OUTDATED
btree_gist	1.7	1.8	OUTDATED
pageinspect	1.12	1.13	OUTDATED
pg_buffercache	1.5	1.6	OUTDATED
pg_stat_statements	1.11	1.12	OUTDATED

```

=====

Quick Summary
=====
postgres:                Total:  4 | Current:  4 | Outdated:  0
gitlabhq_production:     Total: 12 | Current:  6 | Outdated:  5
=====

OVERALL:                  Total: 16 | Current: 10 | Outdated:  5

=====

Quick Fix Commands
=====

# To update all at once, run:
/home/vporalla/pg_extension_script_colors.sh update

# Or copy and run these commands individually:

gitlab-psql -d gitlabhq_production -c 'ALTER EXTENSION pg_buffercache UPDATE;'
gitlab-psql -d gitlabhq_production -c 'ALTER EXTENSION amcheck UPDATE;'
gitlab-psql -d gitlabhq_production -c 'ALTER EXTENSION btree_gist UPDATE;'
gitlab-psql -d gitlabhq_production -c 'ALTER EXTENSION pageinspect UPDATE;'
gitlab-psql -d gitlabhq_production -c 'ALTER EXTENSION pg_stat_statements UPDATE;'

#Show Current Extensions
vporalla@patroni-pgtest-v18-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:~$ /home
=====

PostgreSQL Extensions Report

```

Host: patroni-pgtest-v18-01-db-db-benchmarking

Filter: Current extensions

Date: 2025-10-03 09:18

=====

Database: postgres

Extension	Current	Available	Status
pg_stat_kcache	2.3.1	2.3.1	OK
pg_stat_statements	1.12	1.12	OK
pg_wait_sampling	1.1	1.1	OK
plpgsql	1.0	1.0	OK

Database: gitlabhq_production

Extension	Current	Available	Status
btree_gin	1.3	1.3	OK
dblink	1.2	1.2	OK
pg_trgm	1.6	1.6	OK
pg_wait_sampling	1.1	1.1	OK
pgstattuple	1.5	1.5	OK
plpgsql	1.0	1.0	OK

=====

Quick Summary

=====

postgres: Total: 4 | Current: 4 | Outdated: 0

gitlabhq_production: Total: 12 | Current: 6 | Outdated: 5

=====

OVERALL: Total: 16 | Current: 10 | Outdated: 5

#Update Extensions

vporalla@patroni-pgtest-v18-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:~\$ /home

=====

PostgreSQL Extensions Update

Host: patroni-pgtest-v18-01-db-db-benchmarking

Date: 2025-10-03 09:19

=====

Checking for outdated extensions...

- Database: gitlabhq_production, Extension: pg_buffercache
- Database: gitlabhq_production, Extension: amcheck
- Database: gitlabhq_production, Extension: btree_gist

- Database: gitlabhq_production, Extension: pageinspect
- Database: gitlabhq_production, Extension: pg_stat_statements

Found 5 extension(s) to update.

WARNING: Updating extensions may require application compatibility checks.
It's recommended to backup your databases before proceeding.

Do you want to proceed with the updates? (yes/no): yes

Starting updates...

Updating pg_buffercache in gitlabhq_production... Success
Updated to version: 1.6

Updating amcheck in gitlabhq_production... Success
Updated to version: 1.5

Updating btree_gist in gitlabhq_production... Success
Updated to version: 1.8

Updating pageinspect in gitlabhq_production... Success
Updated to version: 1.13

Updating pg_stat_statements in gitlabhq_production... Success
Updated to version: 1.12

=====
Update Summary
=====
Successful updates: 5

Running post-update status check...

=====
Quick Summary
=====

postgres:	Total: 4 Current: 4 Outdated: 0
gitlabhq_production:	Total: 12 Current: 11 Outdated: 0

OVERALL:	Total: 16 Current: 15 Outdated: 0

#Verify After Update

vporalla@patroni-pgtest-v18-01-db-db-benchmarking.c.gitlab-db-benchmarking.internal:~\$ /home

=====

PostgreSQL Extensions Report

Host: patroni-pgtest-v18-01-db-db-benchmarking

Filter: All extensions

Date: 2025-10-03 09:19

=====

Database: postgres

Extension	Current	Available	Status
pg_stat_kcache	2.3.1	2.3.1	OK
pg_stat_statements	1.12	1.12	OK
pg_wait_sampling	1.1	1.1	OK
plpgsql	1.0	1.0	OK

Database: gitlabhq_production

Extension	Current	Available	Status
pg_repack	1.5.2	1.5.0	↑ NEWER
amcheck	1.5	1.5	OK
btree_gin	1.3	1.3	OK
btree_gist	1.8	1.8	OK
dblink	1.2	1.2	OK
pageinspect	1.13	1.13	OK
pg_buffercache	1.6	1.6	OK
pg_stat_statements	1.12	1.12	OK
pg_trgm	1.6	1.6	OK
pg_wait_sampling	1.1	1.1	OK
pgstattuple	1.5	1.5	OK
plpgsql	1.0	1.0	OK

=====

Quick Summary

=====

postgres:	Total: 4 Current: 4 Outdated: 0
gitlabhq_production:	Total: 12 Current: 11 Outdated: 0

OVERALL:	Total: 16 Current: 15 Outdated: 0
----------	---------------------------------------